

续表

- ④ 数据库设计
- ⑤ 数据库创建和数据加载
- ⑥ 数据库应用软件的功能设计和开发
- ⑦ 数据库应用系统测试
- ⑧ 分析设计和开发文档撰写
- ⑨ 应用软件、数据库和文档提交
- ⑩ 应用系统演示和汇报

实验总结:

① 数据库大作业对学生的要求比较高,不仅要求学生掌握数据库基本原理和方法,数据库设计方法,还需要掌握 Java 或 Python 等数据库开发语言、工具和相应的 Web 开发框架等知识。

② 在数据库课程教学开始就布置该项数据库大作业实验,讲清楚实验内容和要求,以便学生提前准备,学习和熟悉相关的开发语言和工具,这将有利于本实验的完成。

3. 思考题

- ① 数据库应用系统的功能设计对数据库设计有何影响?
- ② 如何协调数据库应用系统的功能设计与数据库设计?
- ③ 在数据库应用开发中,使用的程序设计语言的数据类型与数据库的数据类型如何相互转换?

第 10 章实验 数据库性能监视与调优

数据库性能监视与调优实验分为三个选修实验项目(见表9),它们都属于验证性与设计性相结合的实验。

表9 “数据库性能监视与调优”实验项目一览表

序号	实验项目名称	开设类别	难易程度	建议实验学时	对应《概论》章节
实验 10.1	数据库查询性能调优实验	选修	难	4	第 10 章
*实验 10.2	数据库性能监视实验	选修	难	4	第 10 章
*实验 10.3	数据库系统配置参数调优实验	选修	难	4	第 10 章

实验 10.1 数据库查询性能调优实验

1. 实验内容和要求

理解和掌握数据库查询性能调优的基本原理和方法。学会使用 EXPLAIN 命令分析查询执

行计划、利用索引优化查询性能、优化 SQL 语句；理解和掌握数据库模式规范化设计对查询性能的影响；能够针对给定的数据库模式，设计不同的实例来验证查询性能优化的效果。

实验重点：利用索引优化查询性能、优化 SQL 语句。

实验难点：数据库模式规范化设计对查询性能的影响。

2. 实验报告示例

实验报告			
题目：数据库查询性能调优实验	姓名	日期	
(1) 使用 EXPLAIN 命令查看查询执行计划 查看 Part、PartSupp、Supplier 三个表连接查询的查询执行计划。 <pre> EXPLAIN SELECT * FROM Part P, Partsupp PS, Supplier S WHERE P.Partkey = PS.Partkey AND PS.Suppkey = S.Suppkey AND P.Name = '发动机' ORDER BY S.Acctbal desc, S.Name, P.Partkey; /*该 SQL 语句是要查询零件名为发动机的零件、供应该零件的供应商以及零件供应明细等，并按照供应商的账户余额（降序）、供应商名称（升序）、零件编号（升序）排序输出结果*/ </pre> (2) 利用索引优化查询性能 建立索引，优化 SQL 查询性能。 <pre> CREATE INDEX IDX_part_name ON Part(Name); /*在 Part 表的 Name 属性上建立索引*/ EXPLAIN SELECT * FROM Part P, Partsupp PS, Supplier S WHERE P.Partkey = PS.Partkey AND PS.Suppkey = S.Suppkey AND P.Name = '发动机' ORDER BY S.Acctbal desc, S.Name,P.Partkey; </pre> 比较在 Part 表的 Name 上无索引（任务一）和有索引（任务二）时，两种执行计划有何异同，并实际执行该查询，以验证有索引和无索引时此查询语句的执行性能。 (3) 优化 SQL 语句 ① IN 与 EXISTS 查询。 <pre> SELECT * FROM Orders WHERE Orderkey IN (SELECT Orderkey FROM Lineitem WHERE Partkey IN (SELECT Partkey FROM Part </pre>			

续表

```
WHERE Name = '发动机'));
```

一般地, 使用 EXISTS 查询效率要高于 IN 查询, 改写 SQL 语句如下:

```
SELECT *  
FROM Orders O  
WHERE EXISTS (SELECT *  
FROM Lineitem L  
WHERE O.Orderkey = L.Orderkey AND  
EXISTS (SELECT *  
FROM Part P  
WHERE P.Partkey = L.Partkey  
AND Name = '发动机'));
```

比较两种执行计划, 并实际测试执行性能哪种情况好。

② 尽可能使用不相关子查询, 避免使用相关子查询。

不相关子查询一般比相关子查询执行效率要高。在可能的情况下, 改写相关子查询为不相关子查询, 但是要避免使用 IN 等集合运算符的不相关子查询。

查找这样的订单, 其总价大于该顾客所购商品的平均总价。

相关子查询:

```
SELECT *  
FROM Orders O1  
WHERE O1.Totalprice > (SELECT AVG(O2.Totalprice)  
FROM Orders O2  
WHERE O2.Custkey = O1.Custkey);
```

不相关子查询:

```
SELECT *  
FROM Orders O1, (SELECT O2.Custkey, AVG(O2.Totalprice) AS Avgprice  
FROM Orders O2  
GROUP BY O2.Custkey) AVG1  
/*子查询生成临时派生表, 取名为 AVG1*/  
WHERE O1.Custkey = AVG1.Custkey AND O1.Totalprice > AVG1.Avgprice;
```

比较两种执行计划, 并实际测试执行性能哪种情况好。

(4) 数据库模式规范化设计对查询性能的影响

分析零件供应销售数据库模式中是否存在不规范的设计。该设计在海量数据的情况下查询效率怎样? 如何在设计上进一步提高海量数据的查询效率?

续表

第三范式在一定程度上减少了不必要的冗余,提高了数据库的查询效率,但是如果数据量大且需要大量联合查询的时候,第三范式设计又可能会影响查询效率。零件供应销售数据库模式中存在的规范的设计如下:

① Orders 表中订单的 Totalprice 由 Lineitem 表中该订单的所有订单项的 Extendedprice、Discount 和 Tax 等属性计算得出,即

$$\text{Totalprice} = \text{SUM}(\text{Lineitem.Extendedprice} \times (1 - \text{Lineitem.Discount}) \times (1 + \text{Lineitem.Tax}))$$

该项设计虽然不规范(即定义 Totalprice 为一个冗余数据属性),但大大提高了客户所有订单总金额的查询效率。例如,查询所有购物总金额大于 20 万元的客户编号及其购物总金额:

```
SELECT Custkey, SUM(Totalprice)
FROM Orders
GROUP BY Custkey
HAVING SUM(Totalprice) > 200000.00;
```

如果不用 Totalprice 字段,则 SQL 查询语句为:

```
SELECT O.Custkey, SUM(L.Extendedprice * (1 - L.Discount) * (1 + L.Tax)) AS Sumprice
FROM Orders O, Lineitem L
WHERE O.Orderkey = L.Orderkey
GROUP BY O.Custkey
HAVING Sumprice > 200000.00;
```

② Lineitem 表中订单明细价格 Extendedprice 由 Quantity 和 Part 表中的 Retailprice 得出,即

$$\text{Extendedprice} = \text{Quantity} \times \text{Part.Retailprice}$$

该项设计虽然不规范,但大大提高了客户订单总金额的查询效率。例如,查询所有购物零售总金额(折扣和加税之前的价格,即 Extendedprice)大于 1 万元的订单对应的客户编号及金额:

```
SELECT O.Custkey, SUM(L.Extendedprice) AS Sumextprice
FROM Orders O, Lineitem L
WHERE O.Orderkey = L.Orderkey
GROUP BY O.Custkey
HAVING Sumextprice > 10000.00;
```

如果不使用 Extendedprice,则 SQL 查询为:

续表

```

SELECT O.Custkey, SUM(L.Quantity * P.Retailprice) AS Sumextprice
FROM Orders O, Lineitem L, PartSupp PS, Part P
WHERE O.Orderkey = L.Orderkey AND L.Partkey = PS.Partkey
      AND L.Suppkey = PS.Suppkey AND PS.Partkey = P.Partkey
GROUP BY O.Custkey
HAVING Sumextprice > 10000.00;

```

实验总结:

对于实际数据库应用开发来说,掌握如何利用 SQL 设计数据库查询的技巧是非常必要的。随着数据库数据量的增长,原来有效的 SQL 查询语句的执行效率可能会急剧下降。一个优化的 SQL 查询和一个不好的 SQL 查询,它们的执行效率可能相差十几倍、几十倍,甚至上百倍。

3. 思考题

① 对实验 3.3 中的查询,使用不同的 SQL 语句来表达。比较它们的查询效率,体会并总结查询优化的技巧和方法。

② 利用自动化方法随机产生不同规模的数据集,比较和分析 SQL 查询在不同规模数据集下的执行效率。

实验 10.2 数据库性能监视实验**1. 实验内容和要求**

了解所使用的 DBMS 提供的数据库性能监视功能,学习数据库查询性能监视的基本原理和方法。例如,使用 KingbaseES 的数据库性能监视工具,通过标准统计视图和统计访问函数,查看数据库系统收集到的性能统计信息;运行 ANALYZE 命令来更新数据库统计信息;通过专门工具监视系统的性能。

希望能够熟悉数据库系统有关性能统计信息的标准视图和统计访问函数,了解如何通过系统收集到的统计数据监视系统性能。

实验重点:通过标准统计视图和统计访问函数,查看数据库系统收集到的性能统计信息。

实验难点:分析性能统计信息,找出存在的性能问题。

2. 实验报告示例

实验报告			
题目: 数据库性能监视实验	姓名	日期	
(1) 查看系统标准统计视图和统计访问函数			
① KingbaseES 部分标准统计视图如下:			

续表

a. `sys_stat_activity`: 每个服务器进程一行, 显示数据库、进程、用户、当前查询等信息。只有在打开 `stats_command_string` 参数时, 才能得到报告当前查询的相关信息的各个字段。

b. `sys_stat_database`: 每个数据库一行, 显示数据库、与该数据库连接的活跃服务器进程数、提交的事务总数, 以及在该数据库中回滚数目的总数、读取的磁盘块的总数和缓冲区命中的总数。

c. `sys_stat_all_tables`: 当前数据库中每个表一行, 显示模式和表信息、发起的顺序扫描的总数、顺序扫描抓取的有效数据行的数目、发起的索引扫描的总数、索引扫描抓取的数据行数目, 以及插入、更新和删除的行总数。

d. `sys_stat_all_indexes`: 当前数据库中每个索引一行, 显示输入索引所在表 OID 和索引 OID、模式名、表名和索引名, 包括使用了该索引的索引扫描总数、索引扫描返回的索引记录的数目、使用该索引的简单索引扫描抓取的有效表中的数据行数。

e. `sys_statio_all_tables`: 当前数据库中每个表一行, 显示表、模式和表名, 包含从该表中读取的磁盘块总数、缓冲区命中的次数、在该表上所有索引的磁盘块读取总数和缓冲区命中总数等信息。

f. `sys_statio_all_indexes`: 当前数据库中每个索引一行, 显示表 OID 和索引 OID、模式名、表名和索引名, 该索引的磁盘块读取总数和缓冲区命中的数目。

g. `sys_statio_all_sequences`: 当前数据库中每个序列对象一行, 显示序列的 OID、模式名和序列名、序列磁盘读取总数和缓冲区命中的数目。

② KingbaseES 部分统计访问函数如下:

a. `sys_stat_get_db_numbackends(oid)`: integer, 数据库中活跃的服务器进程数目。

b. `sys_stat_get_db_xact_commit(oid)`: bigint, 数据库中已提交的事务数量。

c. `sys_stat_get_db_xact_rollback(oid)`: bigint, 数据库中回滚的事务数量。

d. `sys_stat_get_tuples_inserted(oid)`: bigint, 插入表中的元组数量。

e. `sys_stat_get_tuples_updated(oid)`: bigint, 在表中已更新的元组数量。

f. `sys_stat_get_tuples_deleted(oid)`: bigint, 从表中删除的元组数量。

g. `sys_stat_get_blocks_fetched(oid)`: bigint, 表或者索引的磁盘块被存取的数量。

h. `sys_backend_pid()`: integer, 附着在当前会话上的服务器进程 ID。

i. `sys_stat_get_backend_pid(integer)`: integer, 给出的服务器进程的进程号。

j. `sys_stat_get_backend_dbid(integer)`: oid, 指定服务器进程的数据库 ID。

k. `sys_stat_get_backend_userid(integer)`: oid, 指定服务器进程的用户 ID。

l. `sys_stat_get_backend_activity(integer)`: text, 服务器进程的当前活跃查询。

m. `sys_stat_get_backend_client_port(integer)`: integer, 连接到给定服务器的客户端端口。

n. `sys_stat_reset()`, boolean, 重置所有当前收集的统计。

续表

(2) 查看数据库管理系统当前活动情况**① 查看当前进程数。**

```
SELECT COUNT(*)  
FROM (SELECT sys_stat_get_backend_idset() AS backendid) AS s;  
/*sys_stat_get_backend_idset() 函数为：获得服务器后台进程集合*/
```

② 查看当前进程的详细活动。

```
SELECT * FROM sys_stat_activity;
```

③ 查看正在进行的 SQL 操作。

```
SELECT sys_stat_get_backend_pid(s.backendid) AS procpid,  
       sys_stat_get_backend_activity(s.backendid) AS current_query  
FROM (SELECT sys_stat_get_backend_idset() AS backendid) AS s;
```

(3) 更新数据库统计信息**① 用 ANALYZE 更新数据库统计信息。**

```
ANALYZE;
```

② 用 ANALYZE 更新数据表的统计信息。

```
ANALYZE Sales.Orders;
```

实验总结：

既要掌握 KingbaseES 有关性能监视和统计分析的系统表和系统视图，以及相应的性能分析和监视语句；又要掌握利用合适的数据库性能监视工具进行数据库性能监视。

3. 思考题

- ① 试总结你所用的 DBMS 的性能监视工具的功能和使用方法。
- ② 总结和分析数据库性能监视主要监视哪些数据库性能指标。一旦数据库性能指标出现异常，应当如何进行处理？

实验 10.3 数据库系统配置参数调优实验*1. 实验内容和要求**

了解数据库系统级参数与连接级参数的配置及调优的基本原理和方法。了解如何通过修改这些参数设置来调整系统运行时的配置，以优化系统性能。

了解并熟悉数据库各级参数的作用以及配置，包括系统级参数配置和调优、数据库级参数配置和调优、会话（连接）级参数配置和调优。

实验重点：数据库级参数配置和调优、连接级参数配置和调优。

实验难点：系统级参数配置和调优。

2. 实验报告示例

实验报告			
题目：数据库系统配置参数 调优实验	姓名	日期	
(1) 显示参数的命令 ① 显示 XXX 参数。 SHOW shared_buffers; /*专门的显示系统会话值的命令，可以显示所有参数的值*/ ② 显示 xxx 参数。 SELECT shared_buffers; /*只能显示一个参数的值*/ (2) 设置会话级参数 ① 设置提交延迟时间参数为 10 s，对本次会话有效。 SET SESSION COMMIT_DELAY to 10; ② 设置提交延迟时间参数为 10 s，对本次提交事务有效。 SET LOCAL COMMIT_DELAY to 10; (3) 修改数据库默认参数 设置 test 数据库的密码长度为 10 个字符，该设置覆盖数据库的默认设置，对该数据库上启动的每个会话都有效。 ALTER DATABASE test SET password_length TO 10; (4) 设置系统级或全局级参数配置 ① 关闭自动清理存储空间开关，自动修改数据库系统配置文件，该修改将在重启数据库服务器后才生效。 ALTER SYSTEM SET autovacuum = off; ② 设置数据库监听的地址，自动修改数据库系统配置文件，该修改将在重启数据库服务器后才生效。 ALTER SYSTEM SET listen_addresses = '127.0.0.1'; (5) 优化系统级参数 优化 KingbaseES 数据库管理系统的 shared_buffers 参数。			

续表

```
ALTER SYSTEM SET shared_buffers = 80000;
```

该参数是很重要的参数,数据库管理系统通过 `shared_buffers` 与内核和磁盘打交道。一方面,该参数应该设置得足够大以应对通常的表访问操作,让更多的数据缓存在 `shared_buffers` 中。通常设置为实际 RAM 的 10%,例如,设置 80 000 个缓冲区(每个缓冲区一般为 8 KB,8 万个缓冲区大小为 625 MB)。将所有的内存都给 `shared_buffers`,则会导致没有内存来运行程序;另一方面,该参数应该设置得足够小,以避免操作系统因缺少内存而将页面换进/换出。

(6) 优化会话级参数

① 优化 KingbaseES 数据库管理系统的 `work_mem` 参数。

```
SET work_mem = 61440;
```

在执行排序操作时,系统会根据 `work_mem` 的大小,决定是否将一个大的结果集拆分为几个小的和 `work_mem` 差不多大小的临时文件。显然,拆分的结果会降低排序的速度。因此增加 `work_mem` 的大小会有助于提高排序速度,通常设置为实际 RAM 的 2%~4%;另外还要根据需求排序结果集的大小而定,比如 61 440 (60 MB)。

② 优化 KingbaseES 数据库管理系统的 `temp_buffers` 参数。

```
SET temp_buffers = 2048; /*设置临时缓冲区为 2 GB*/
```

该参数称为临时缓冲区,用于数据库会话访问临时表数据,系统默认值为 8 MB。可以在单独的会话中对该参数进行设置,尤其是需要访问比较大的临时表时,这项优化将会带来显著的性能提升。

实验总结:

数据库系统配置参数一般由数据库管理员负责修改。修改数据库系统配置需要非常小心谨慎,一旦修改不当可能会导致数据库系统性能下降,甚至系统崩溃。

3. 思考题

- ① 试通过一个实际示例,测试不同的 `temp_buffers` 大小对临时表查询性能的影响。
- ② 总结系统级参数和连接级参数的区别与联系。

第 11 章实验 数据库备份与恢复

数据库备份与恢复实验分为三个选修实验项目(见表 10)。这些实验项目均为验证性实验。